


***UNIVERSIDAD MAYOR DE SAN ANDRÉS***

***FACULTAD DE INGENIERÍA***

***INGENIERÍA ELECTRÓNICA***

# *Sistemas de control II - A. Técnicas de control*

# *[ETN-902 - ETN-1034]*

# *Bitácora Control – Planta de Procesamiento*

***Integrantes ETN-902:***

*Ajnota Cahuaya Limber*

*Gonzales Estrada Kevin Rodrigo*

*Gómez Orellana Karel Walter*

***Docente: Ing. Jorge Antonio Nava Amador***

***Auxiliares: Rhem Josafat Huarachi Machaca, Miguel Huanca Mita***

***La Paz - Bolivia***

# Análisis para el par variador-motor

**Índice**

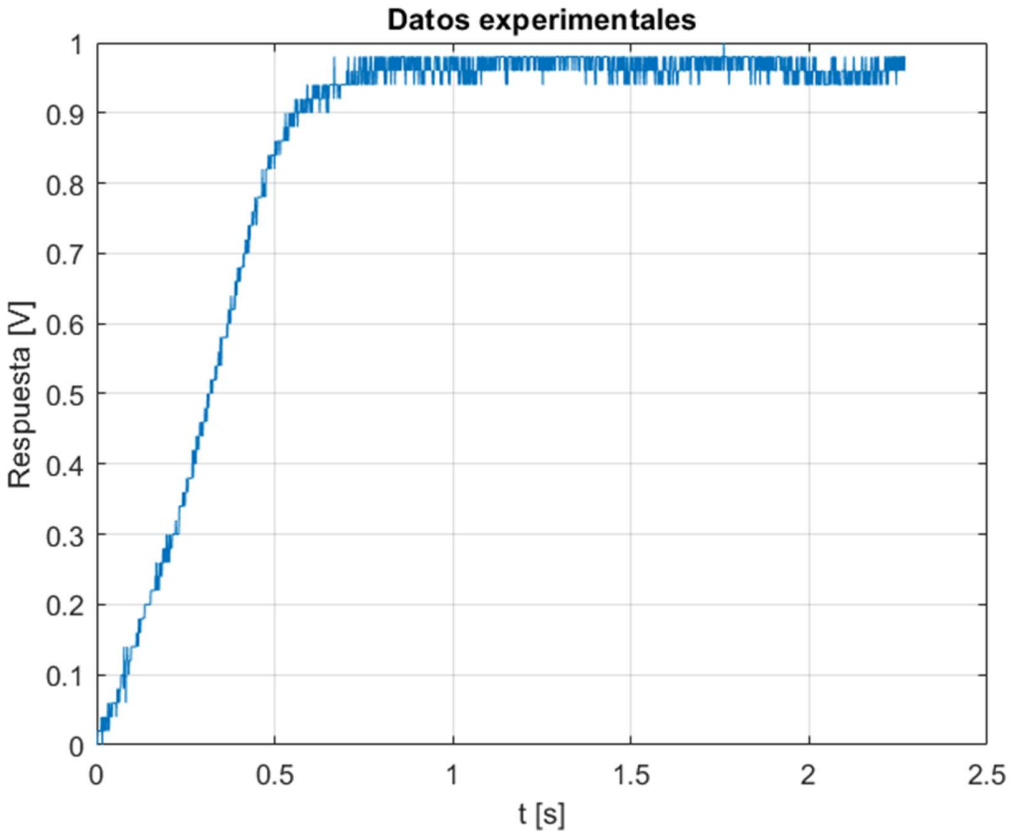

Análisis para el par variador-motor
1. Datos experimentales
2. Discretización
3. Función de transferencia a lazo cerrado
4. Trabajo de analisis y diseño analitico
4.1 Especificaciones de desempeño
4.2 Obtención del controlador continuo
4.3 Discretización del controlador
4.4 Pruebas del rechazo a perturbaciones en SIMULINK

5. Programación del PLC
5.1 Ecuación en recurrencia del controlador
5.2 Programa LADDER del PLC
5.3 Influencia del tiempo de cálculo del PLC
6. Pruebas experimentales y validacion del diseño
6.1 Datos experimentales obtenidos


## 1. Datos experimentales

Para el par variador-motor, se tienen los siguientes datos experimentales obtenidos:

```
out_V = Source;
vscale=5e-01;
Ts=1e-03;
inicio=1731;
Vout = out_V*vscale/25;
Vout = Vout(inicio:length(Vout));
t_vector(1)=0;
for i=2:length(Vout)
    t_vector(i) = t_vector(i-1) + Ts;
end
plot(t_vector,Vout)
grid on
title('Datos experimentales')
xlabel('t [s]')
ylabel('Respuesta [V]')
```




A estos datos experimentales se debe aplicar un escalamiento para obtener los datos en unidades para la velocidad angular (rad/s). Para ello empezamos con la relación existente entre revoluciones por minuto y la frecuencia del encoder incremental que se utilizó par realizar las mediciones en la ecuación (1):

$$R.P.M = 0.15 * f_{\text{encoder}}\,[\text{Hz}] \quad \ldots(1)$$

Y dado que la relación de revoluciones por minuto a velocidad angular es:

$$\omega\left[\frac{\text{rad}}{s}\right] = \frac{\pi}{30} * R.P.M. \quad \ldots(2)$$

Obteniendo entonces la relación de frecuencia en Hz del encoder a velocidad angular, mostrada en la ecuacion (3) a continuación:

$$\omega\left[\frac{\text{rad}}{s}\right] = \frac{\pi}{200} * f_{\text{encoder}}\,[\text{Hz}] \quad \ldots(3)$$

Pero, para obtener una curva de respuesta temporal típica, se utilizó un circuito conversor de frecuencia a voltaje. Este circuito tiene una relación de entrada en frecuencia y salida en voltaje que se muestra en la ecuacion (4):

$$\text{Vout}\,[V] = 8.36\text{x}10^{-4} * f_{\text{encoder}}\,[\text{Hz}] \quad \ldots(4)$$

Que puede escribirse como sigue:

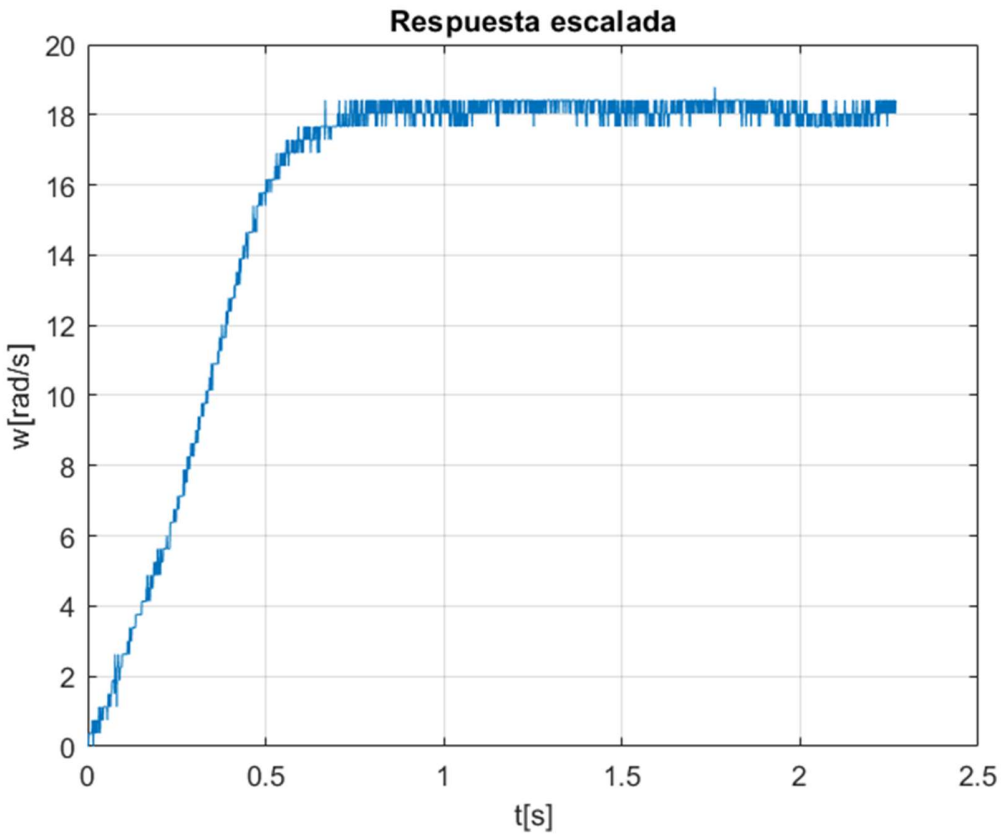
$$f_{\text{encoder}}\,[\text{Hz}] = 1.1962\text{x}10^{3} * \text{Vout}\,[V] \quad \ldots(5)$$

Utilizando la ecuación (3), que relaciona la velocidad angular y la frecuencia del encoder, con la ecuación (5), que relaciona la frecuencia del encoder con la tensión medida en el osciloscopio, obtenemos:

$$\omega\left[\frac{\text{rad}}{s}\right] = \frac{\pi}{200} * 1.1962\text{x}10^{3} * \text{Vout}\,[V] \quad \ldots(6)$$

Mediante la ecuación (6) podemos realizar un escalamiento de la respuesta que se midió con el osciloscopio:

```
vel_angular = Vout*pi*1.1962e3/200;
plot(t_vector,vel_angular)
grid on
title('Respuesta escalada')
xlabel('t[s]')
ylabel('w[rad/s]')
```




Utilizando la respuesta ya escalada, y mediante el *toolBox* de MATLAB *"SystemIdentification"* o *"Ident"*, se pudo estimar la función de transferencia que se muestra a continuación:

```
f_mot=tf(tf2)
```

```
f_mot =

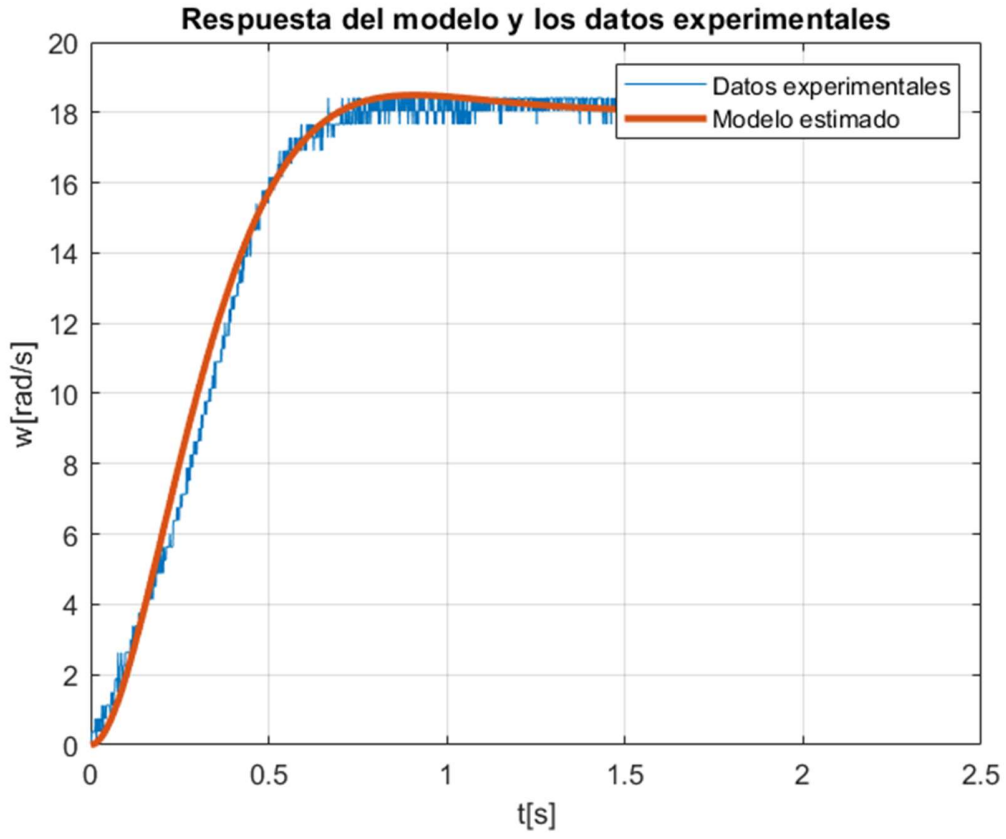
  From input "u1" to output "y1":
          48.55
  ---------------------
  s^2 + 8.259 s + 29.02

Name: tf2
Continuous-time transfer function.
```

```
[y,x]=step(10.8*f_mot);
```

Comparando la respuesta de la función de transferencia con los datos experimentales se tiene lo siguiente:

```
plot(t_vector,vel_angular)
grid on
title('Respuesta del modelo y los datos experimentales')
xlabel('t[s]')
ylabel('w[rad/s]')
hold on
plot(x,y,'lineWidth',2.5)
hold off
legend('Datos experimentales','Modelo estimado')
```



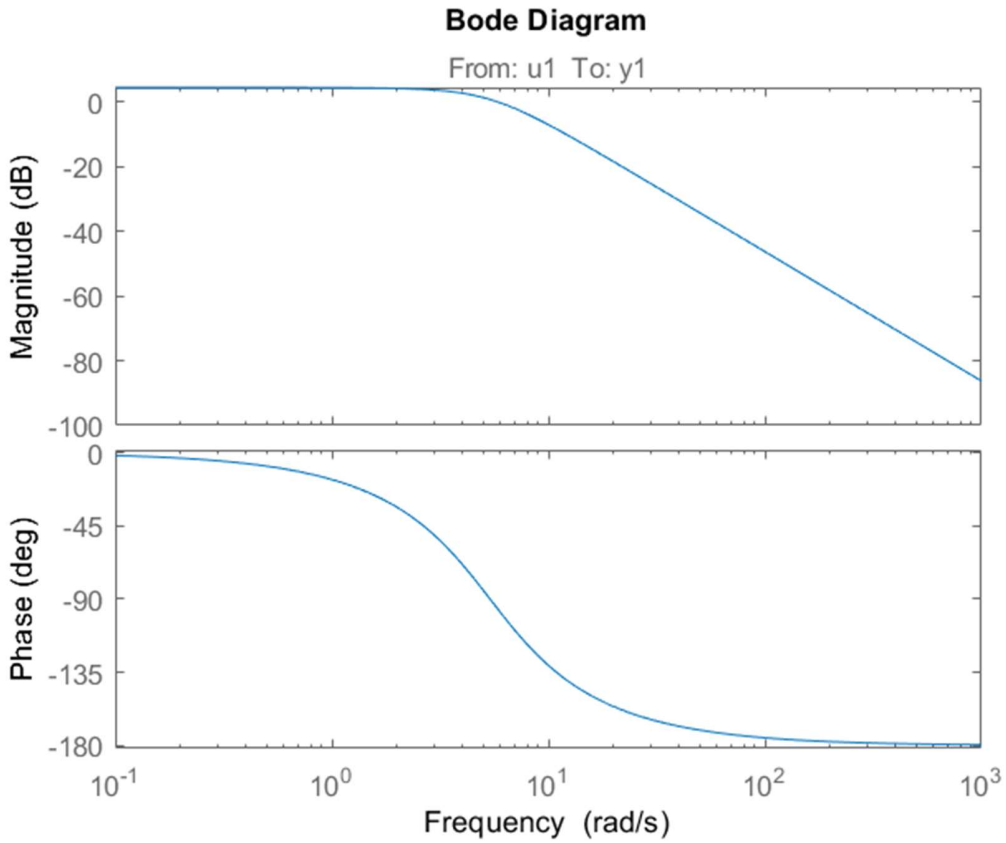

Los datos de la respuesta temporal del modelo son los siguientes:

```
stepinfo(f_mot)
ans = struct with fields:
```

```
      RiseTime: 0.4355
  SettlingTime: 1.0282
   SettlingMin: 1.5135
   SettlingMax: 1.7123
     Overshoot: 2.3499
    Undershoot: 0
          Peak: 1.7123
      PeakTime: 0.9033
```

## 2. Discretización

```
f_mot=tf(tf2);
w=logspace(-1,3,500);
bode(f_mot,w)
```




Aproximadamente se puede ver que el ancho de banda es de 5 [rad/s]. Sin embargo, podemos obtener el ancho de banda con más precisión con la siguiente instrucción de MATLAB:

```
B=bandwidth(f_mot)
```
```
B = 4.9300
```

Se puede obtener un periodo de muestreo mediante el siguiente criterio aproximado:

$$\omega_s = N_m * \omega_m \qquad 20 \leq N_m \leq 50$$

Donde $\omega_m$ es el ancho de banda $y$ $N_m$ es un número natural

```
Ts = 1/(50*(B/(2*pi)))
```
```
Ts = 0.0255
```

Lo que es aproximadamente:

```
Ts = 20e-3;
```

Sin embargo, para poder obtener una respuesta lo mas fiel posible se opta por usar el siguiente periodo de muestreo:

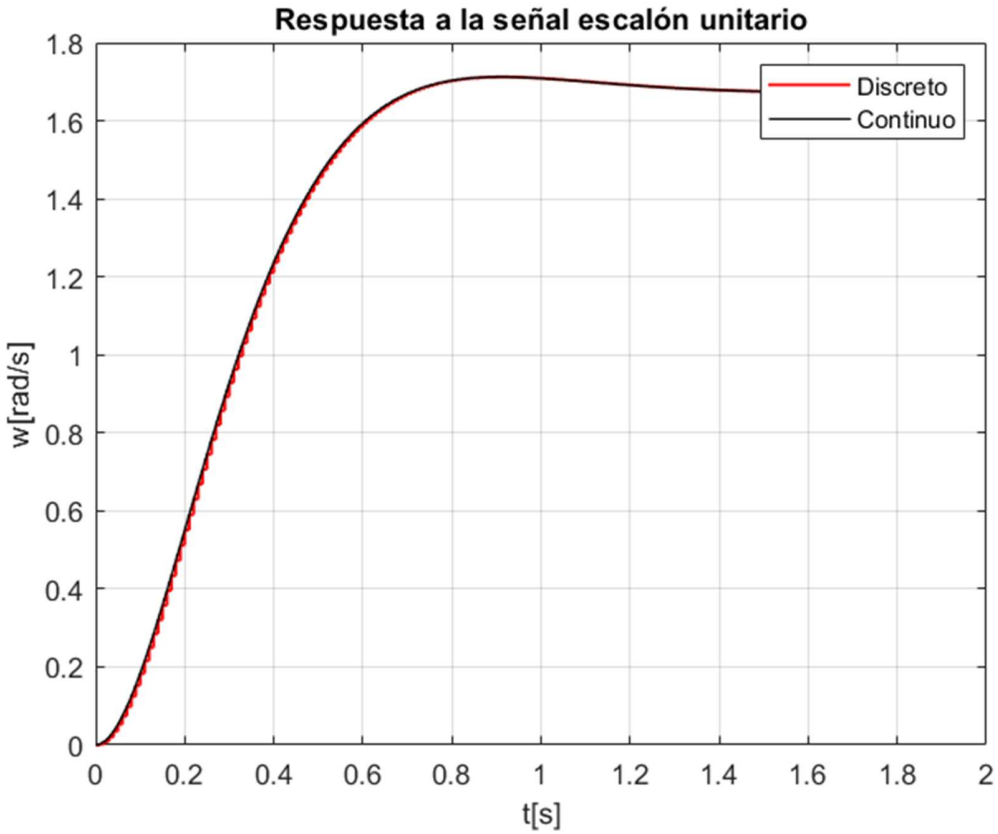
```
Ts = 10e-3;
```

Con este periodo de muestreo podemos realizar una discretización utilizando un retenedor de orden cero como se muestra con la siguiente instrucción de MATLAB:

```
mot_d=c2d(f_mot,Ts,'zoh')
```
```
mot_d =

  From input "u1" to output "y1":
  0.002361 z + 0.002297
  ----------------------
  z^2 - 1.918 z + 0.9207

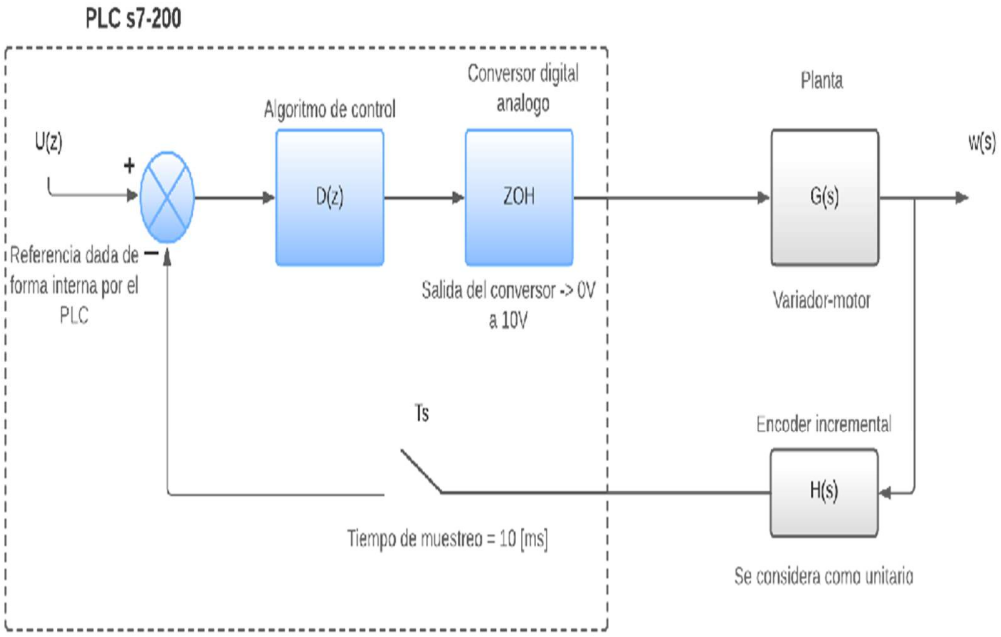
Sample time: 0.01 seconds
Discrete-time transfer function.
```
```
[y,x]=step(mot_d);
stairs(x,y,'r','lineWidth',1.1)
hold on
[y,x]=step(f_mot);
plot(x,y,'black','lineWidth',0.8)
hold off
title('Respuesta a la señal escalón unitario')
xlabel('t[s]')
ylabel('w[rad/s]')
grid on
legend('Discreto','Continuo')
```




```
stepinfo(mot_d)
```

```
ans = struct with fields:
        RiseTime: 0.4400
    SettlingTime: 1.0300
     SettlingMin: 1.5203
     SettlingMax: 1.7124
       Overshoot: 2.3507
      Undershoot: 0
            Peak: 1.7124
        PeakTime: 0.9100
```

## 3. Función de transferencia a lazo cerrado




De este diagrama de bloques podemos obtener la siguiente función de transferencia a lazo cerrado:

$$\frac{\Omega_{(z)}}{U_{(z)}} = \frac{D_{(z)}\overline{\mathrm{GZOH}_{(z)}}}{1 + D_{(z)}\overline{\mathrm{HGZOH}_{(z)}}} \quad \ldots(7)$$

Y para simplificar tomamos el bloque de la planta y del retenedor de orden cero como uno solo, ademas, se tomara la realimentacion H(z) como unitaria:

$$\frac{\Omega_{(z)}}{U_{(z)}} = \frac{D_{(z)}G_{(z)}}{1 + D_{(z)}G_{(z)}} \quad \ldots(8)$$

## 4. Trabajo de analisis y diseño analitico

Para poder realizar el análisis, se debe obtener la función de transferencia a lazo cerrado reemplazando datos en la ecuación (8) y omitiendo el controlador D(z) por el momento:

```
[num,den]=tfdata(mot_d);
num=cell2mat(num)
```

```
num = 1×3
         0    0.0024    0.0023
```

```
den=cell2mat(den)
```

```
den = 1×3
    1.0000   -1.9179    0.9207
```

```
A=abs(num(2));
B=abs(num(3));
C=abs(den(1));
D=abs(den(2));
```

```
E=abs(den(3));
Mz = tf([0 A B],[C A-D E+B],Ts)
```
```
Mz =

  0.002361 z + 0.002297
  ---------------------
  z^2 - 1.916 z + 0.923

Sample time: 0.01 seconds
Discrete-time transfer function.
```

## 4.1 Especificaciones de desempeño

Para el sistema que deseamos controlar, las especificaciones de desempeño vienen dadas sobre todo en el tiempo de la respuesta transitoria, ya que se necesita que la cinta transportadora llegue lo más rápido posible al régimen estacionario o a su respuesta estacionaria, esto ya que es a esta velocidad de régimen que se pretende que los cilindros se mueven y que los cilindros neumáticos los empuje en el momento correcto. Es por esto que se da un tiempo razonablemente veloz de asentamiento de 0.25 [s], esto hace que el factor de amortiguamiento relativo tenga que ser un poco menor (sobre pico mayor), este reduccion sera dada de forma arbitria como zeta = 0.5, que es menor que le factor original de la funcion de transferencia (zeta=0.76) .

Los requerimientos serán:

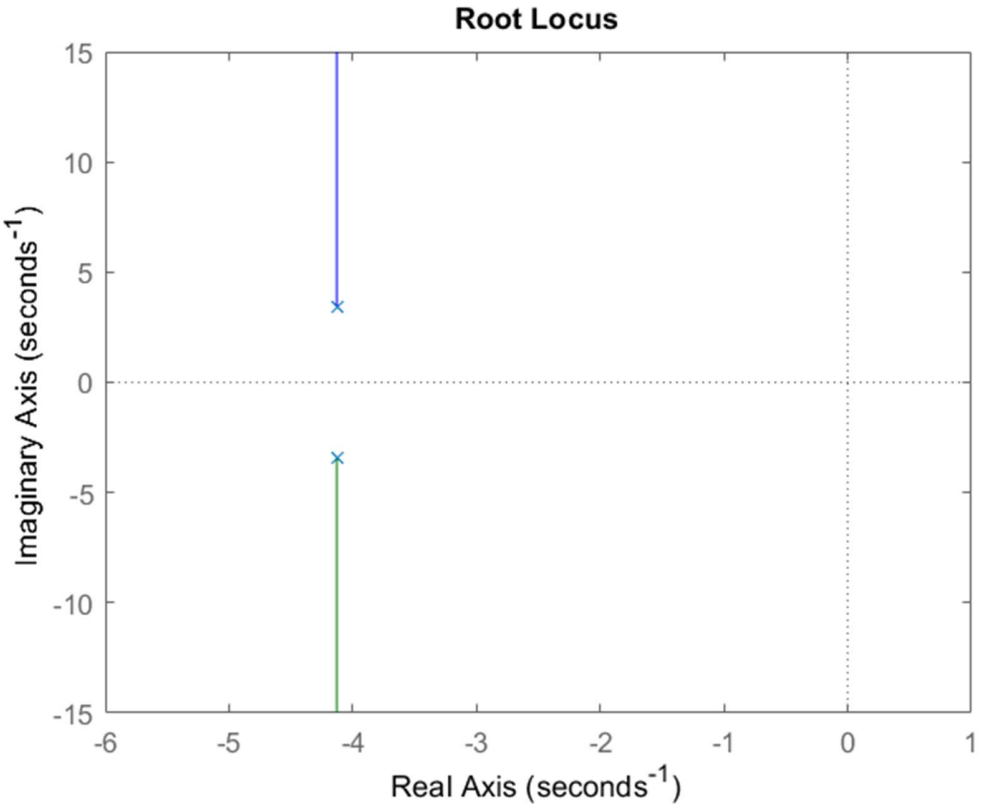
$$\xi = 0.5$$
$$t_s = 0.25\,[s]$$

## 4.2 Obtención del controlador continuo

Con estos requerimientos se debe diseñar un controlador, en este caso un controlador PID. La forma de diseñar el controlador será, en primera instancia, diseñar un controlador en tiempo continuo, el cual será realizado por el método del lugar de las raíces. Posteriormente, el controlador será discretizado.

Entonces obtenemos el grafico del lugar de las raices de la función a lazo abierto:

```
rlocus(f_mot)
```




Primero obtenemos los datos que son necesarios para realizar el diseño, mediante el tiempo de asentamiento y el factor de amortiguamiento, hallamos sigma y la frecuencia natural no amortiguada de los polos deseados (que serán dominantes):

$$t_s = \frac{4}{\sigma} = \frac{4}{\zeta \omega_n}$$

Obtenemos que:

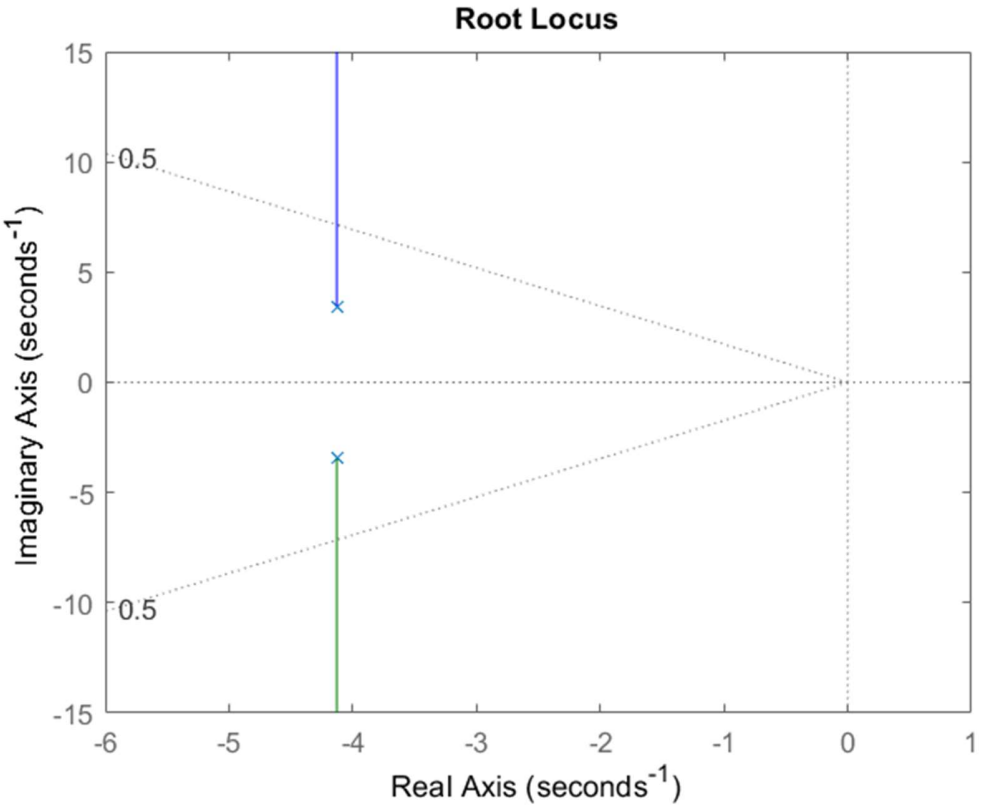
$$\sigma \approx 12 \quad y \quad \omega_n \approx 24 \left[\frac{\text{rad}}{s}\right]$$

Con esto hallamos la frecuencia natural amortiguada:

$$\omega_d = 20.78 \left[\frac{\text{rad}}{s}\right]$$

Con esto podemos revisar el lugar de las raices:

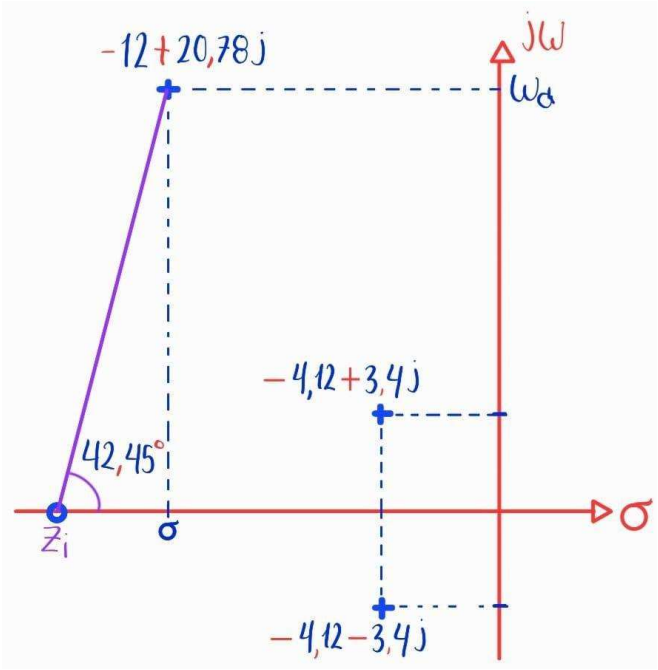
```
rlocus(f_mot)
sgrid(0.5,24)
```




Se puede observar fácilmente que no se puede compensar el sistema únicamente con la ganancia. Entonces iniciamos con el diseño del controlador PID, empezaremos por el control PD ideal, esto añadiendo un cero:

$$\mathrm{Dd}(s) = s + z_i$$

Ahora, ubicar los polos reales de la planta y uno de los polos deseados ($p_d = \sigma \pm \omega_d$). Y se debe hallar la suma de los ángulos de los polos con respecto al polo deseado. Siendo esta suma de $222^o$, con esto calculamos la contribución requerida por el cero que se va a añadir es de $42,45^o$.




Con el gráfico, y mediante trigonometría, se encuentra que la posición del cero que es:

$$z_i = 34.74$$

$$\mathrm{Dd}(s) = s + 37.74$$

Podemos comprobar que el cero permite mover el lugar de las raíces para el factor de amortiguamiento y frecuencia natural no amortiguada que deseábamos, esto mediante el lugar de las raíces:

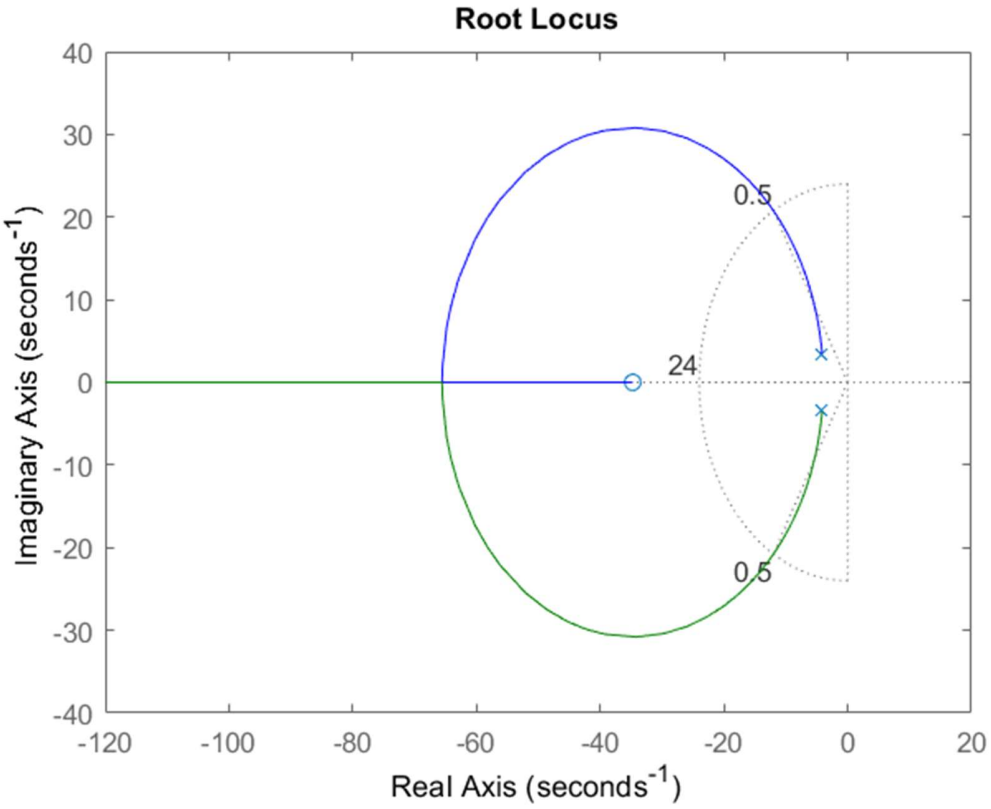
```
DDs=tf([1 34.74],[0 1])
```

```
DDs =

  s + 34.74

Continuous-time transfer function.
```

```
rlocus(DDs*f_mot)
sgrid(0.5,24)
```




Comprobando así que el lugar de las raíces se modificó como queríamos, ahora se debe agregar el control PI ideal, esto añadiendo un cero cercano al origen (arbitrario) y un polo en el origen:

$$\mathrm{DI}(s) = \frac{s + z_j}{s}$$

Se puede probar con el cero en 0.5. Con esto tenemos el controlador casi completo:

```
DIs = tf([1 0.5],[1 0])
DIs =

  s + 0.5
  -------
     s

Continuous-time transfer function.
PID = DIs*DDs
PID =

  s^2 + 35.24 s + 17.37
  ---------------------
            s

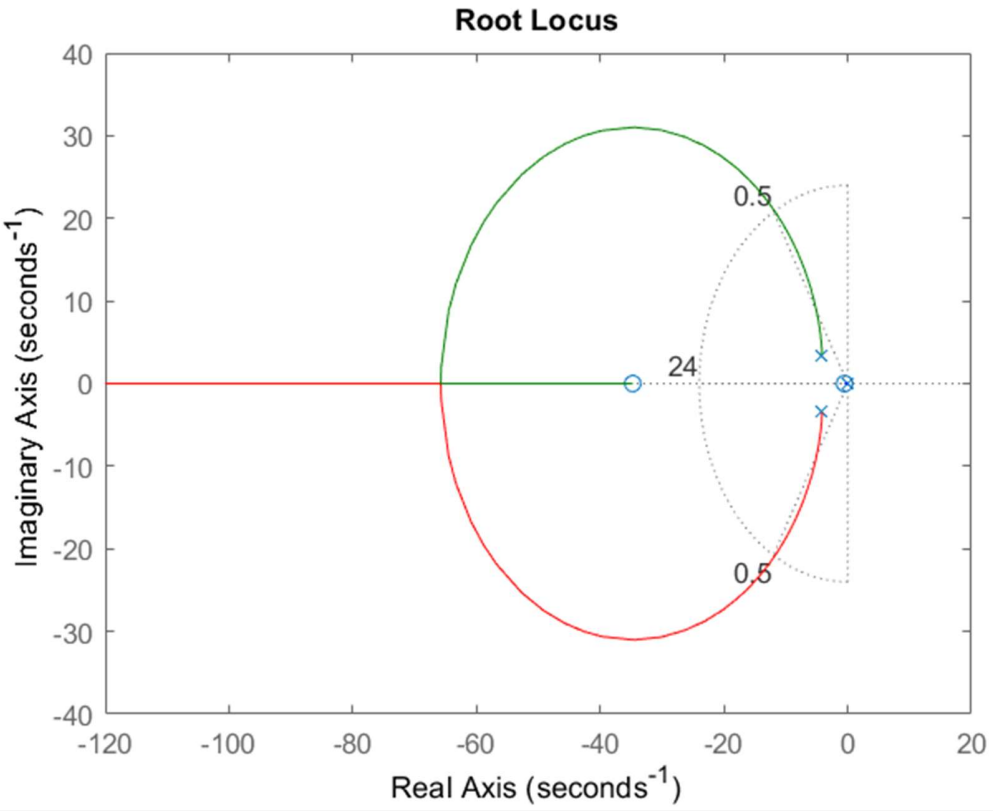
Continuous-time transfer function.
```

Sólo falta hallar la ganancia que corresponde, para esto nos valemos de la condición de módulo, obteniendo asi que la ganancia K es:

```
K=0.312;
```

Podemos probar una vez mas el lugar de las raices, ya con el controlador PID:

```
rlocus(PID*f_mot)
sgrid(0.5,24)
```



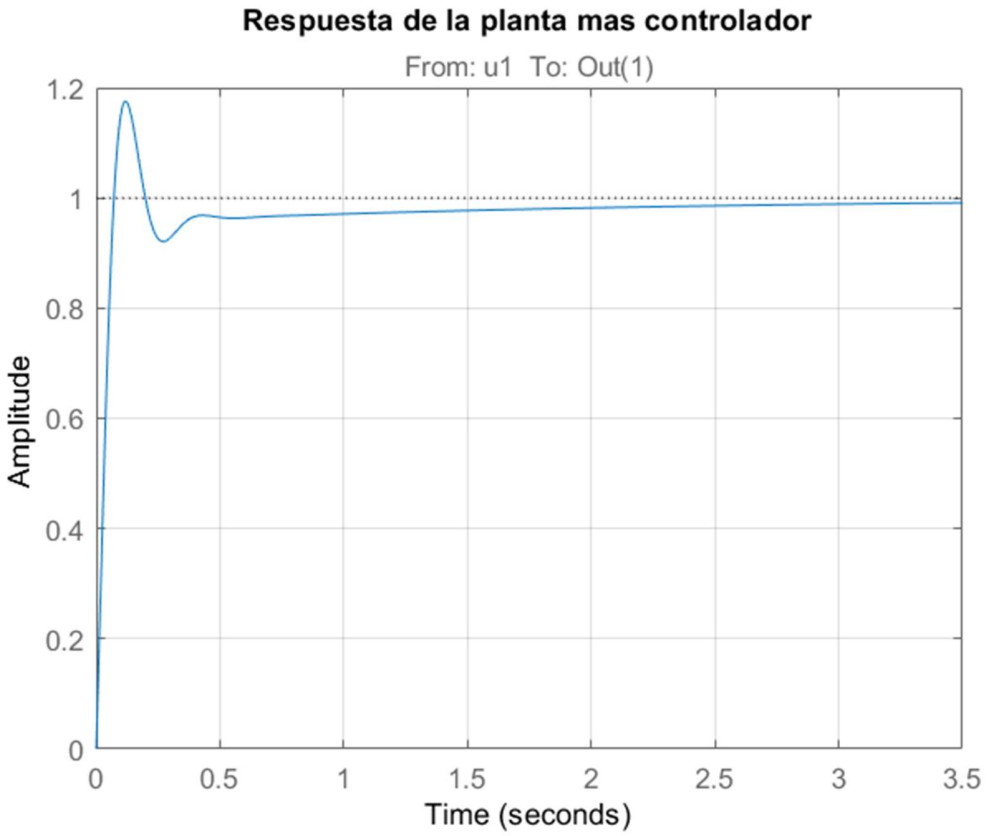

```
lazo_cont = feedback(K*PID*f_mot,1)
```

```
lazo_cont =

  From input "u1" to output:
     15.15 s^2 + 533.8 s + 263.1
  ---------------------------------
  s^3 + 23.41 s^2 + 562.8 s + 263.1

Continuous-time transfer function.
```

```
step(lazo_cont)
grid on
title('Respuesta de la planta mas controlador')
```




Finalmente se obtienen las constantes del controlador:

```
Kp = 11.52;
Ti = 2.028;
Td = 28.39e-3;
```

### 4.3 Discretización del controlador

Ya con el controlador en tiempo continuo realizado, se puede realizar la discretización, para esto solo hace falta elegir una discretización de muchas que hay para el PID. En este caso, utilizamos una discretizacion simple, con la discretización mediante el operador derivador:

```
Ki = Kp*Ts/Ti;
Kd = Kp*Td/Ts;
z=tf('z',Ts);
Dz = Kp + Ki*z/(z-1) + Kd*(z-1)/z
```

```
Dz =

  44.28 z^2 - 76.93 z + 32.71
  ---------------------------
           z^2 - z

Sample time: 0.01 seconds
Discrete-time transfer function.
```

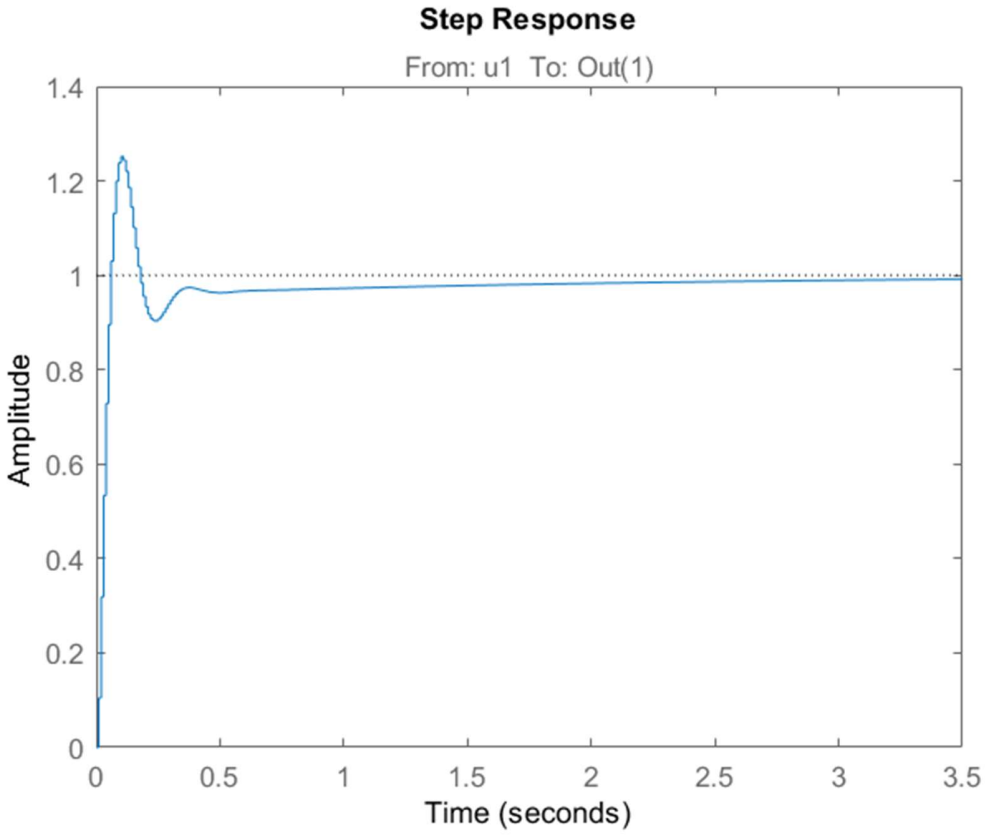
```
lazo_disc = feedback(Dz*mot_d,1)
```

```
lazo_disc =
```

```
  From input "u1" to output:
  0.1046 z^3 - 0.07993 z^2 - 0.0995 z + 0.07513
  ---------------------------------------------
  z^4 - 2.813 z^3 + 2.759 z^2 - 1.02 z + 0.07513

Sample time: 0.01 seconds
Discrete-time transfer function.
```

```
step(lazo_disc)
```




Se tiene el problema de que la acción derivativa no es físicamente realizable, se utilizará la aproximación de filtro de primer orden, aquí se debe utilizar un valor de *alpha* arbitrario entre 0.05 y 0.1. Para poder decidir cual tiene un menor efecto sobre la respuesta antes mostrada, se simularán 6 valores de alpha:

```
alpha = 0.05;
KD = Kp*2*Td/(Ts+2*alpha*Td);
cd = (Ts-2*alpha*Td)/(Ts+2*alpha*Td);
Dz = Kp + Ki*z/(z-1) + KD*(z-1)/(z+cd)
```

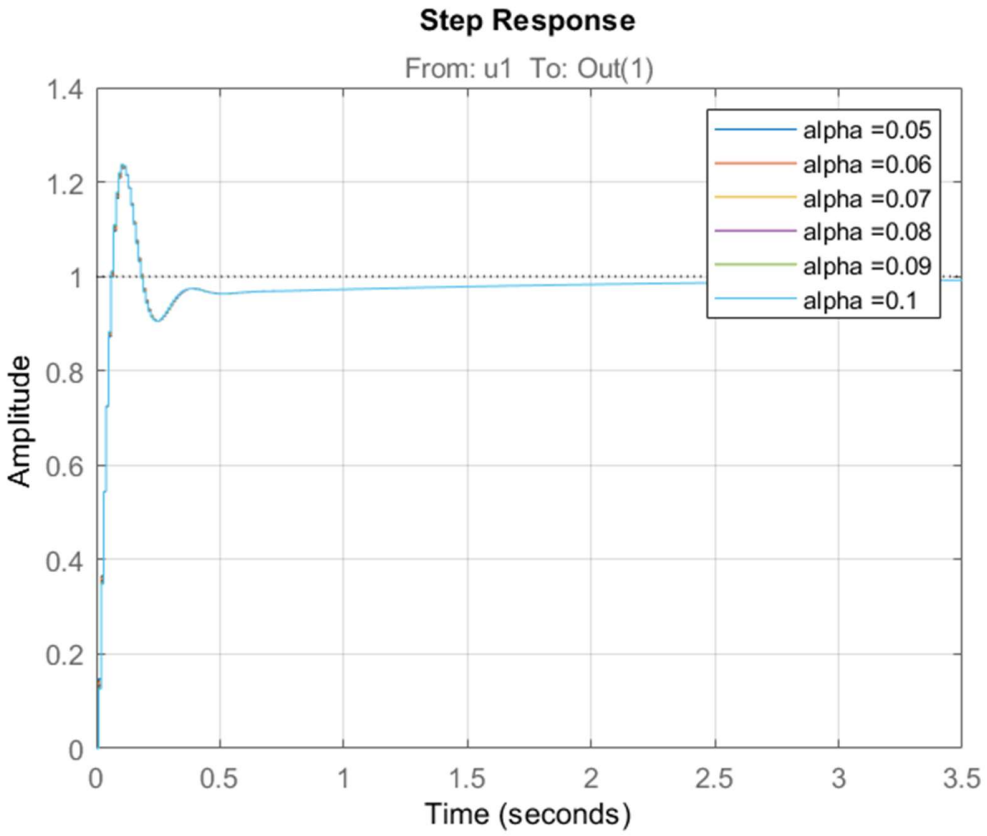
```
Dz =

  62.52 z^2 - 107 z + 44.52
  -------------------------
   z^2 - 0.4422 z - 0.5578

Sample time: 0.01 seconds
Discrete-time transfer function.
```

```
step(feedback(Dz*mot_d,1))
```

```
grid on
hold on
lege = zeros(1,5);
for alpha=0.06:0.01:0.1
    KD = Kp*2*Td/(Ts+2*alpha*Td);
    cd = (Ts-2*alpha*Td)/(Ts+2*alpha*Td);
    Dz = Kp + Ki*z/(z-1) + KD*(z-1)/(z+cd);
    step(feedback(Dz*mot_d,1),3.5);
end
hold off
legend('alpha =0.05','alpha =0.06','alpha =0.07','alpha =0.08','alpha
=0.09','alpha =0.1');
```




```
alpha = 0.08;
KD = Kp*2*Td/(Ts+2*alpha*Td);
cd = (Ts-2*alpha*Td)/(Ts+2*alpha*Td);
Dz = Kp + Ki*z/(z-1) + KD*(z-1)/(z+cd)
```

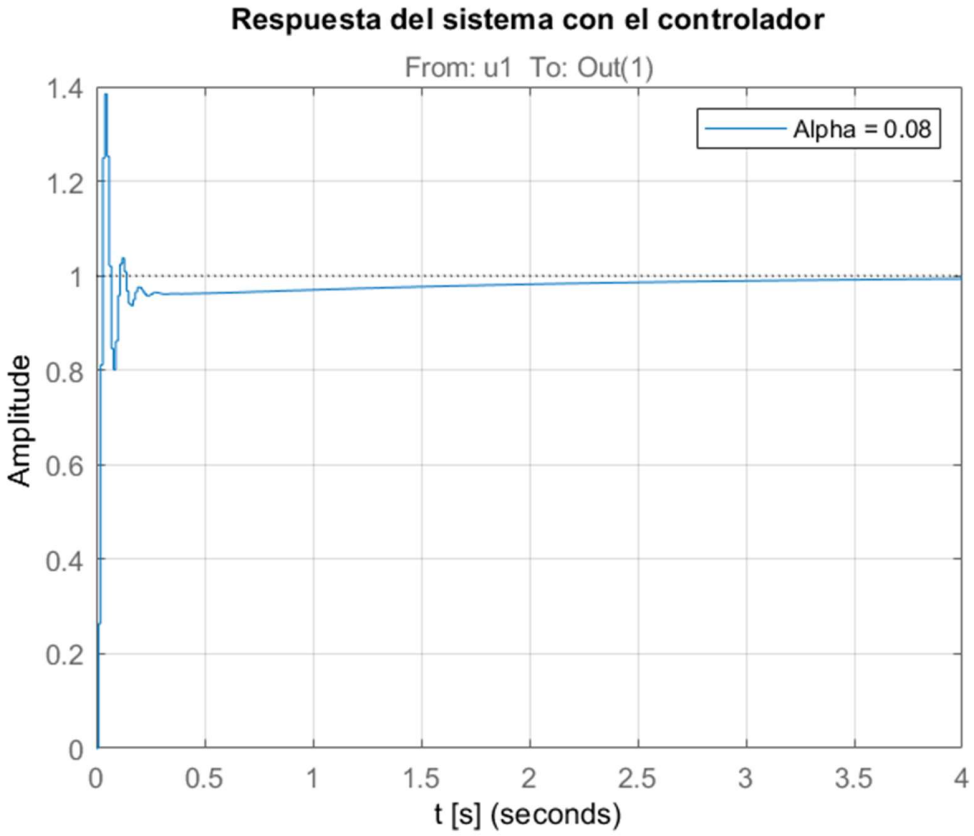
```
Dz =

  111.6 z^2 - 216 z + 104.5
  -------------------------
   z^2 - 1.389 z + 0.3886

Sample time: 0.01 seconds
Discrete-time transfer function.
```

```
step(feedback(Dz*mot_d,1))
title('Respuesta del sistema con el controlador')
```

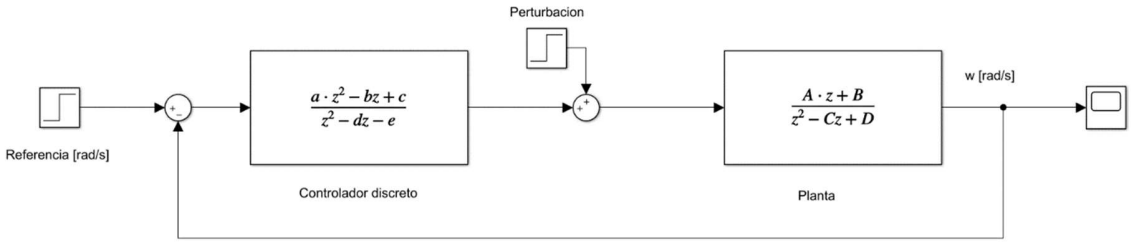
```
xlabel('t [s]')
legend('Alpha = 0.08')
grid on
```



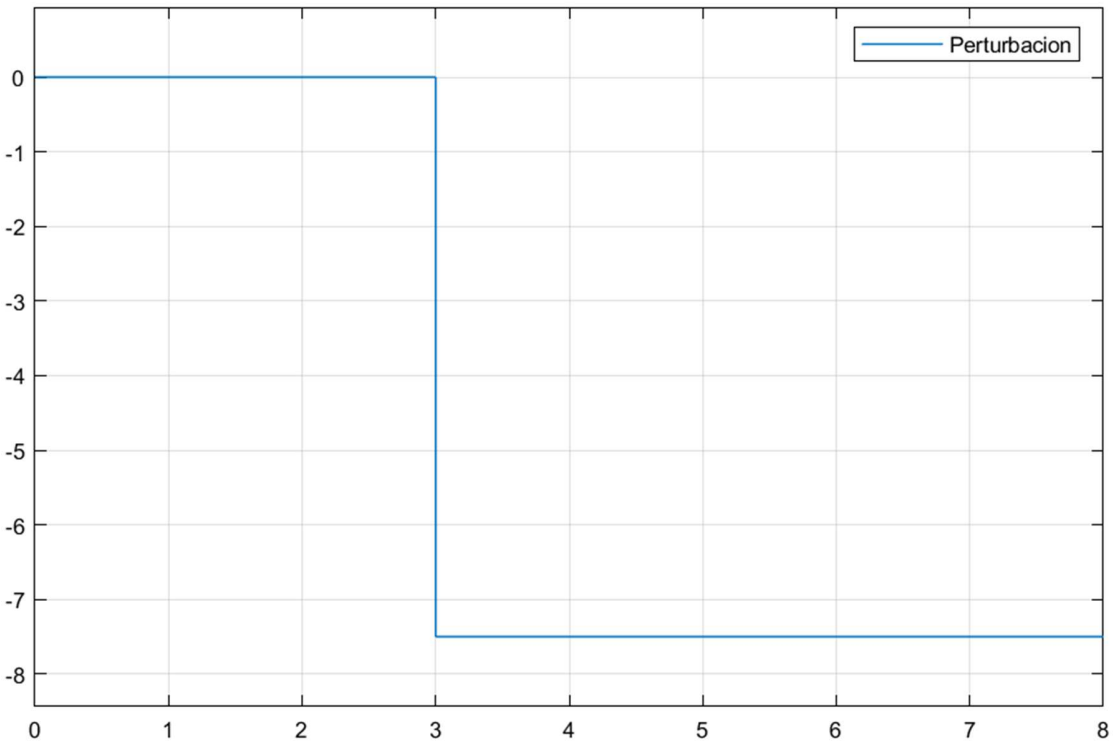

```
stepinfo(feedback(Dz*mot_d,1))
ans = struct with fields:
        RiseTime: 0.0200
    SettlingTime: 1.8100
     SettlingMin: 0.8010
     SettlingMax: 1.3843
       Overshoot: 38.4260
      Undershoot: 0
            Peak: 1.3843
        PeakTime: 0.0400
```

### 4.4 Pruebas del rechazo a perturbaciones en SIMULINK

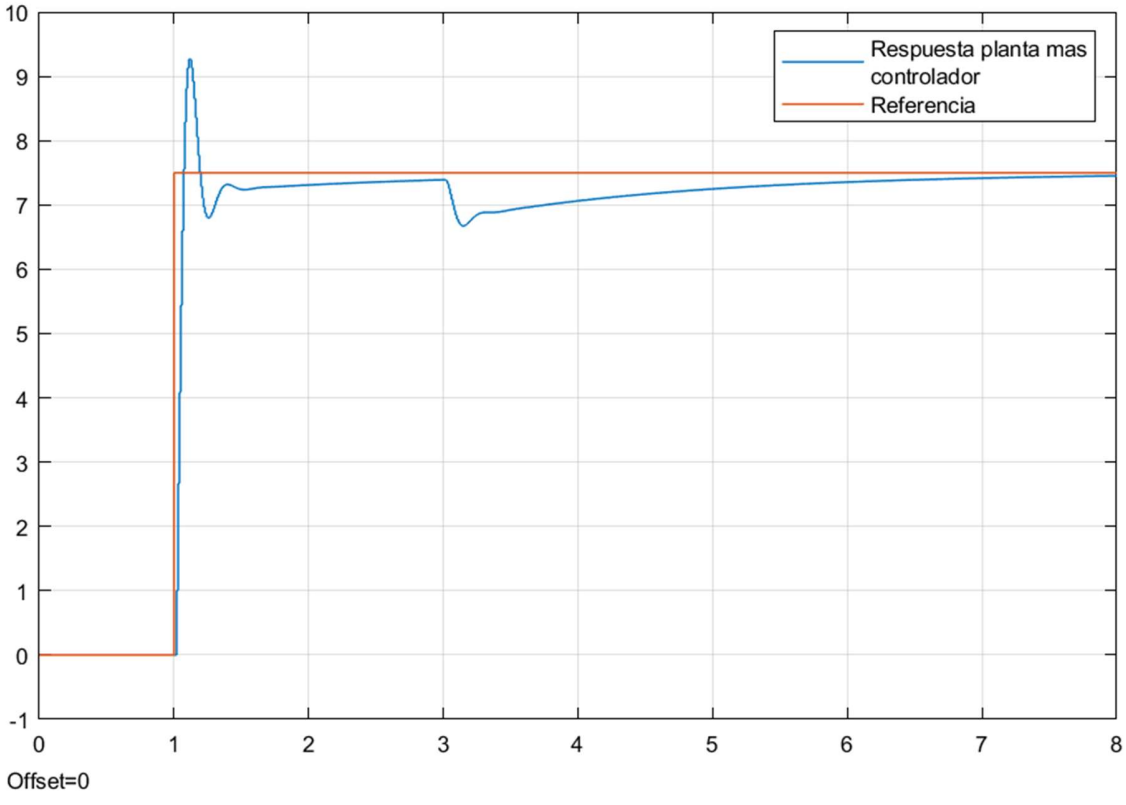
Para comprobar el rechazo a perturbaciones usamos SIMULINK.




Agregando una funcion paso como perturbacion:




La respuesta del sistema mas el controlador sera el siguiente:



Como se puede ver el controlador diseñado tiene cierta capacidad para realizar rechazo a perturbaciones.

# 5. Programación del PLC

## 5.1 Ecuación en recurrencia del controlador

Teniendo ya la función de transferencia discreta del controlador se puede, a partir de esta, obtener la ecuación en recurrencia que nos permite realizar la programación del mismo en el PLC (o en cualquier otro computador digital).

A partir de la función de transferencia obtenemos entonces la ecuación en recurrencia:

$$y(k) = 0.6247y(k-1) + 0.3753y(k-2) + 56.56e(k) - 97.137e(k-1) + 40.66e(k-2)$$

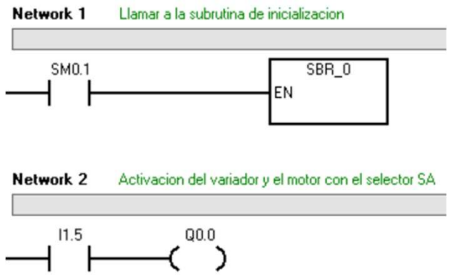
## 5.2 Programa LADDER del PLC

El programa se divide en tres partes:

- El programa principal.
- La subrutina de inicialización
- La subrutina de interrupción temporizada

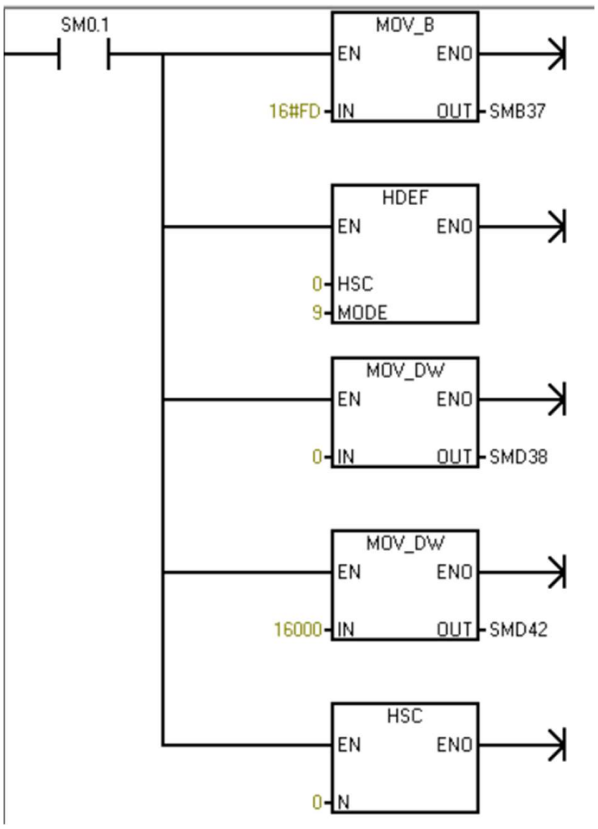
A continuación se explican las tres partes:

***Programa principal***, es la parte del programa que se ejecuta en cada ciclo del PLC. En el programa principal se llama a la rutina de inicialización, mediante la marca especial ***SM0.1*** que solo se activa en el primer ciclo de funcionamiento del PLC. En el programa esta también la activación del variador y el motor (Q0.0) mediante una entrada, en este caso el selector SA (I1.5).

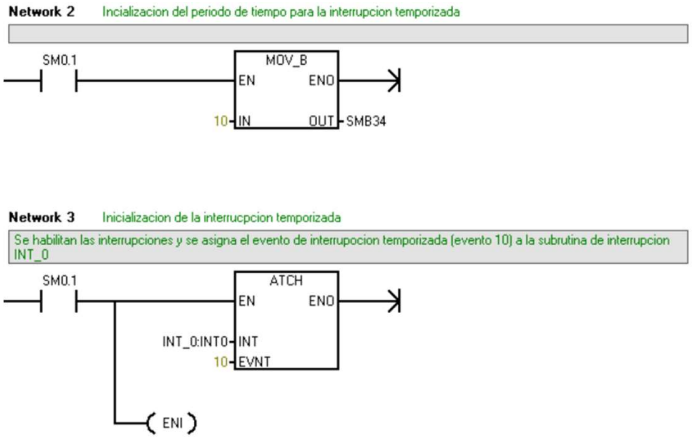



***Subrutina de inicialización***, esta subrutina, como su nombre indica, inicializa todas las variables necesarias para la ejecución del programa, además de inicializar el contador rápido HSC0 (el contador rápido es con el cual se leen los pulsos del encoder).

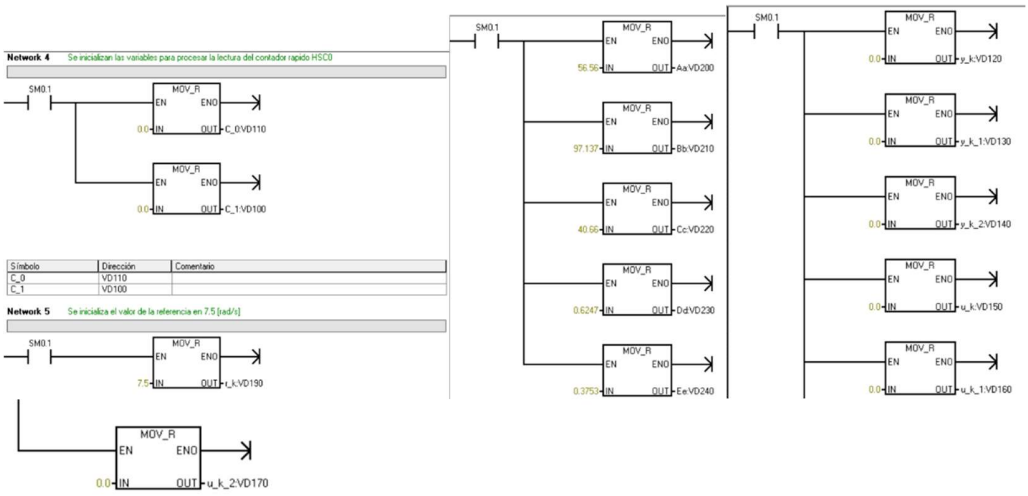
La subrutina configura y habilita el contador rápido HSC0, la configuración se carga moviendo un byte específico en la posición de memoria ***SMB37***. Posteriormente se define el contador HSC0 en el modo 9 (modo de encoder con dos fases, A y B). Se sigue con la configuración del valor inicial del contador y algún valor del contador que active una interrupción (esta opción no será utilizada en esta ocasión, sin embargo igual se la debe configurar). Finalmente, se habilita el contador rápido HSC0.




En la subrutina también se inicializa la interrupción temporizada, primero se configura el periodo de funcionamiento de la interrupción (entre 1 [ms] y 255[ms]) moviendo el valor a la posición de memoria ***SMB34***. Luego se asigna el evento de la interrupción (evento 10) a la subrutina de interrupción ***INT_0***, finalmente, se habilitan las interrupciones.



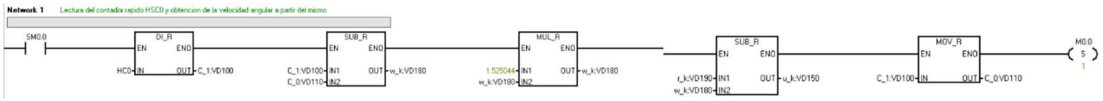

Por último, se inicializan como tal todas las variables que se usan en el programa, como la referencia; los valores actual y pasado del contador rápido; y los coeficientes de la ecuación en recurrencia.



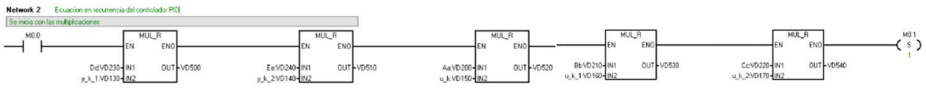

Para procesar la información del contador rápido, y obtener la velocidad angular, se usa la siguiente ecuación:

$$w(t) = \frac{C_1 - C_2}{10\text{ms}} * \frac{1000\text{ ms}}{1s} * \frac{1\text{ rev}}{412\text{ pasos}} * \frac{2\pi\text{ rad}}{1\text{ rev}}$$

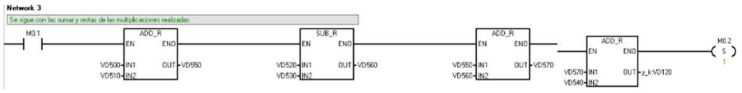
Siendo el programa:



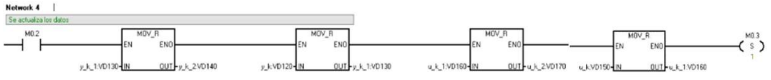

Para la ecuación en recurrencia se multiplica primero los coeficientes por las variables correspondientes:



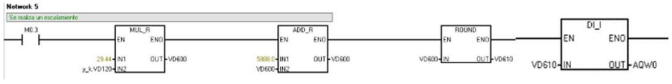

Luego se realizan las sumas y las restas necesarias:

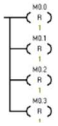



Se deben actualizar los valores de las variables que utiliza la ecuación en recurrencia:




Finalmente se realiza un escalamiento mediante una recta que tome en cuenta los valores máximos y mínimos de la salida del controlador; y el máximo y mínimo que espera recibir el conversor análogo-digital del PLC (el conversor espera valores entre 0 y 32000 para una salida entre 0 VDC y 10 VDC):





### 5.3 Influencia del tiempo de cálculo del PLC

Con la información del manual se pueden aproximar el tiempo que lleva al PLC ejecutar todo el programa. Se obtuvo que el tiempo de ejecución es de aproximadamente 2.1 [ms], que es un tiempo considerable, por lo que se realizará el análisis de la influencia de este tiempo en la respuesta del sistema más el controlador. Se agrega entonces un retardo de transporte (que puede ser tratado con la transformada Z modificada).

```
s=tf('s');
mot_d_delay = c2d(f_mot*exp(-s*2.1e-3),10e-3)
```

```
mot_d_delay =

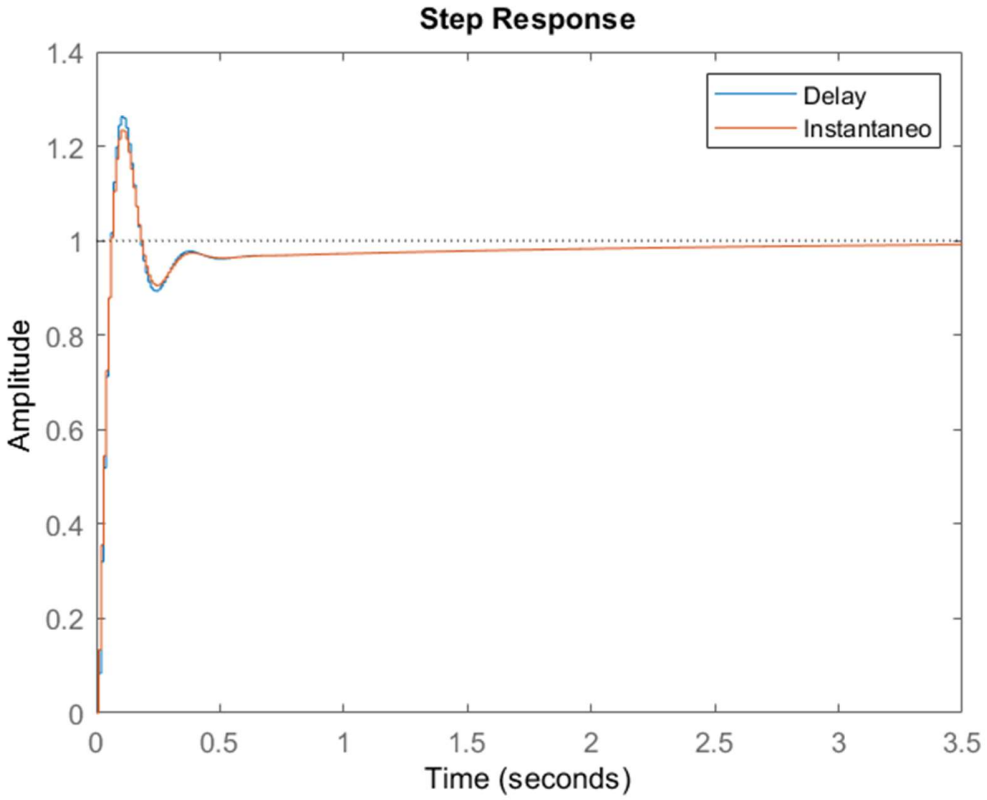
  From input to output "y1":
          0.001482 z^2 + 0.003077 z + 9.913e-05
  z^(-1) * -------------------------------------
                  z^2 - 1.918 z + 0.9207

Sample time: 0.01 seconds
Discrete-time transfer function.
```

```
step(feedback(Dz*mot_d_delay,1))
```

```
Warning: Use the state-space representation for more efficient modeling of discrete-time
delays.
```

```
hold on
step(feedback(Dz*mot_d,1))
hold off
legend('Delay','Instantaneo')
```



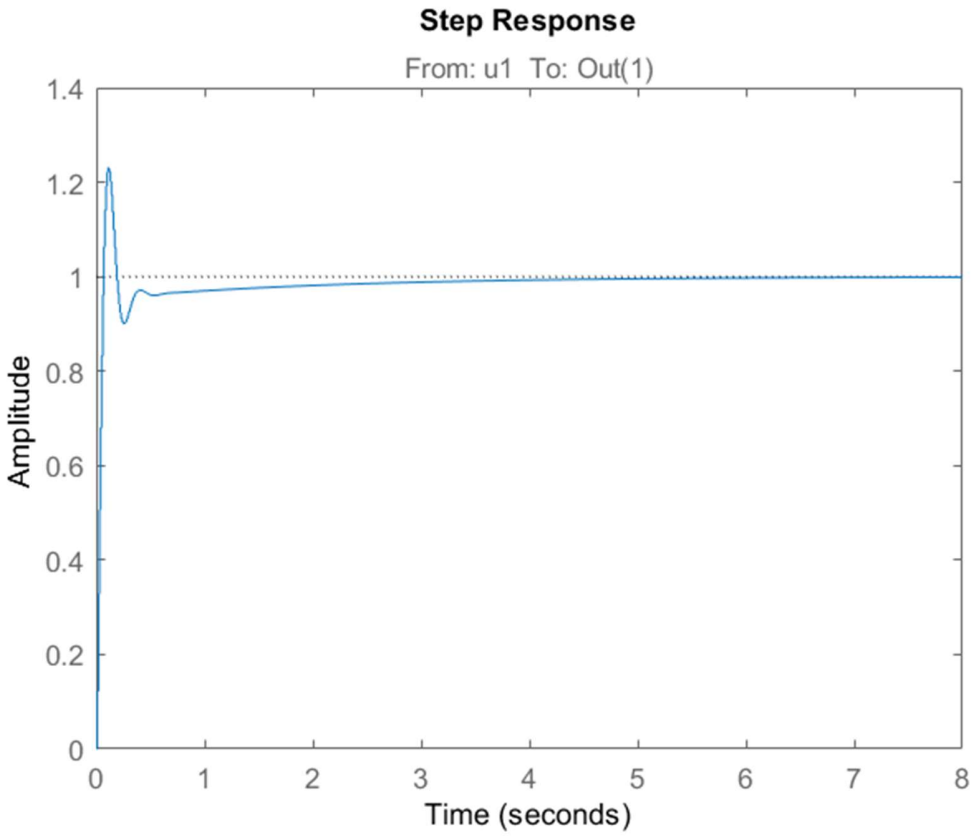

```
Kpp = 10.6;
Ti = 2.028;
Td = 28.39e-3;
KDD = Kpp*2*Td/(Ts+2*alpha*Td);
Kii = Kpp*Ts/Ti;
Dzz = Kpp + Kii*z/(z-1) + KDD*(z-1)/(z+cd)
```

```
Dzz =

  52.04 z^2 - 89.38 z + 37.41
  ---------------------------
    z^2 - 0.6247 z - 0.3753

Sample time: 0.01 seconds
Discrete-time transfer function.
```

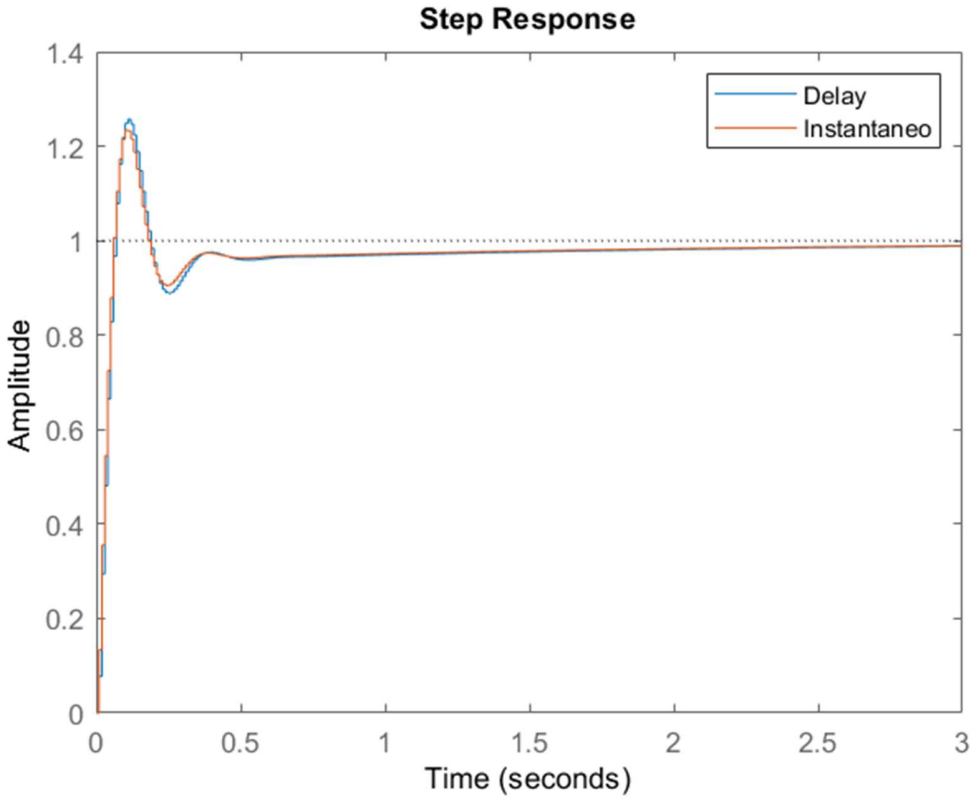
```
step(feedback(Dzz*mot_d,1))
```




```
step(feedback(Dzz*mot_d_delay,1),3)
```

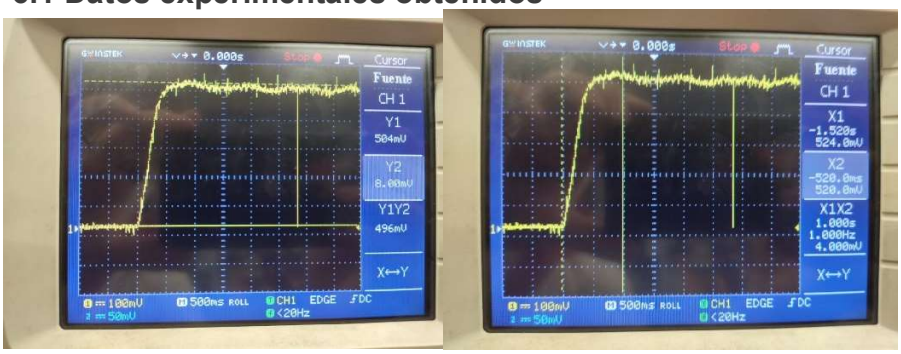
```
Warning: Use the state-space representation for more efficient modeling of discrete-time
delays.
```

```
hold on
step(feedback(Dz*mot_d,1),3)
hold off
legend('Delay','Instantaneo')
```

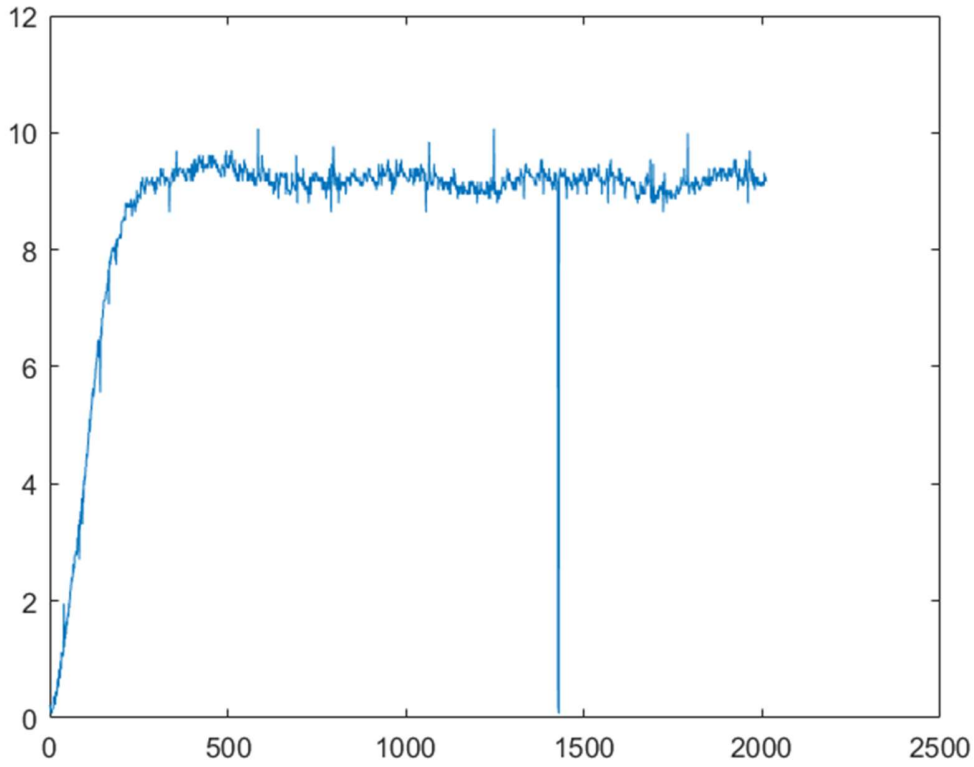



# 6. Pruebas experimentales

## 6.1 Datos experimentales obtenidos



```
Ts=2e-3;
Vscale_saldia = 1e-1;
salida = SourcePID(1988:length(SourcePID));
sal_PID = (salida*Vscale_saldia/25)*pi*1.1962e3/200;;
t=0;
for t=1:length(sal_PID)
    t
end
plot(sal_PID)
```




```
ref = 7.5*ones(1,length(sal_PID));
ref = ref';

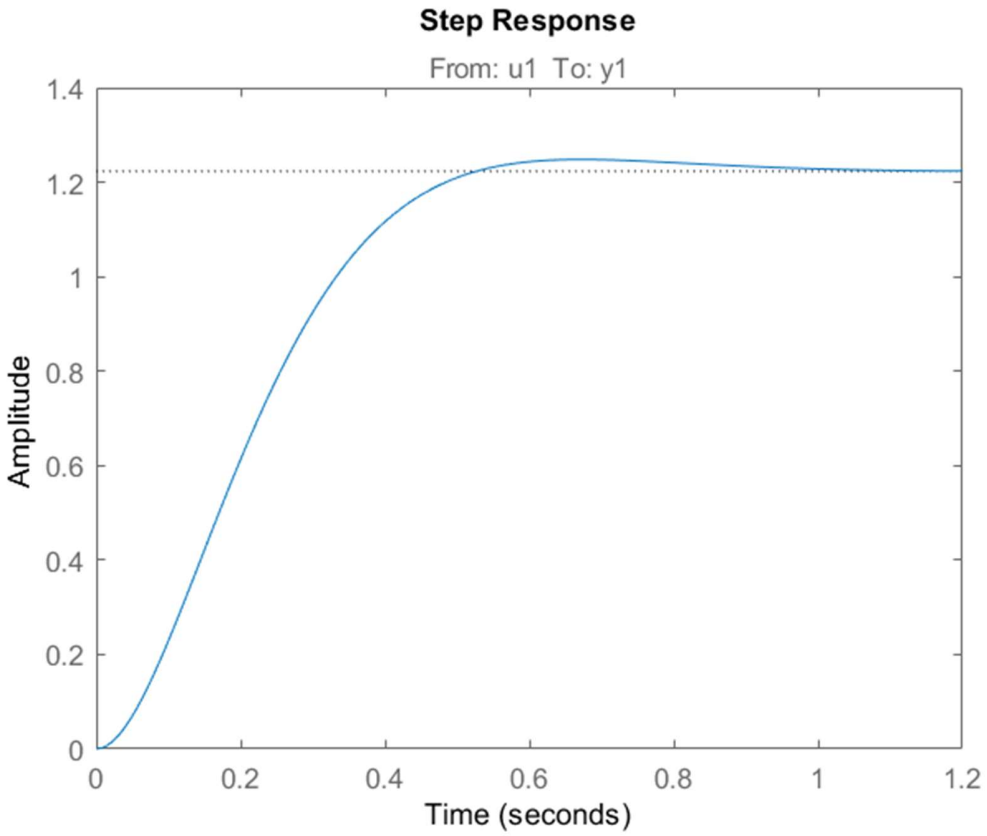
salida_fin = tf(tf_sal)
```

```
salida_fin =

  From input "u1" to output "y1":
         68.02
  --------------------
  s^2 + 11.62 s + 55.6

Name: tf_sal
Continuous-time transfer function.
```

```
step(salida_fin)
```




```
stepinfo(salida_fin)
ans = struct with fields:
        RiseTime: 0.3207
    SettlingTime: 0.6858
     SettlingMin: 1.1112
     SettlingMax: 1.2481
       Overshoot: 2.0103
      Undershoot: 0
            Peak: 1.2481
        PeakTime: 0.6736
```

## 6.2 Conclusiones

- El controlador se pudo realizar, pero tiene un alto costo computacional, en este caso (PID) tal vez seria mas recomendable utilizar el PID que incluye el propio PLC, aunque este sea del tipo autoajustable.
- Los resultados no llegaron a ser los estimados por la parte teórica, a consecuencia de las protecciones del propio variador de frecuencia, como rampas de aceleración.
- Se realizo una adaptación de señal que nos permitía realizar la adquisición de datos, por medio del osciloscopio, esta adaptación se realizo mediante un circuito conversor de frecuencia a voltaje.

### 6.2 Recomendaciones

- A la hora de realizar el diseño, se vio más sencillo el realizar un diseño de un controlador continuo y luego la discretización de este. Esto ya que el diseño directo en discreto puede llegar a ser ligeramente mas complicado.